Program 1: University Scholarship Allocation

1. Objectives

- To check students' scholarship eligibility.

- To consider aggregate marks, subject marks, attendance, and family income.

- To display eligible students in ranked order.

2. Algorithm

1. Start the program.

2. Enter student details.

3. Check the eligibility conditions.

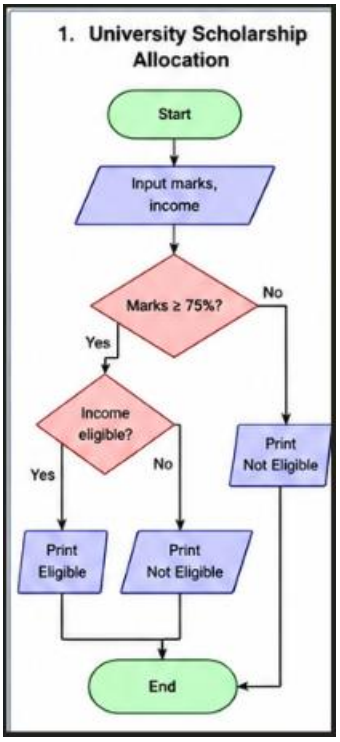
4. Add eligible students to a list.

5. Sort eligible students by aggregate marks, attendance, and income.

6. Display the eligible students.

7. Stop.

3.flowchart



4.Pseudocode

START

INPUT student details

FOR each student

IF aggregate >= 75 AND subject >= 60

AND attendance >= 75 AND income <= 250000

ADD student to eligible list

END IF

END FOR

SORT eligible students

DISPLAY eligible students

STOP

5. Python Code

```
students = [
   {"name": "Asha", "aggregate": 88, "subject": 75,
    "attendance": 91, "income": 180000},
   {"name": "Balu", "aggregate": 88, "subject": 70,
    "attendance": 85, "income": 150000},
   {"name": "Divya", "aggregate": 82, "subject": 65,
    "attendance": 80, "income": 220000},
   {"name": "Ravi", "aggregate": 70, "subject": 65,
    "attendance": 80, "income": 180000}
]


eligible = []


for student in students:
   if (student["aggregate"] >= 75
         and student["subject"] >= 60
         and student["attendance"] >= 75
         and student["income"] <= 250000):
      eligible.append(student)


eligible.sort(key=lambda s: (
   -s["aggregate"], -s["attendance"], s["income"]
```

))

print("ELIGIBLE SCHOLARSHIP APPLICANTS")

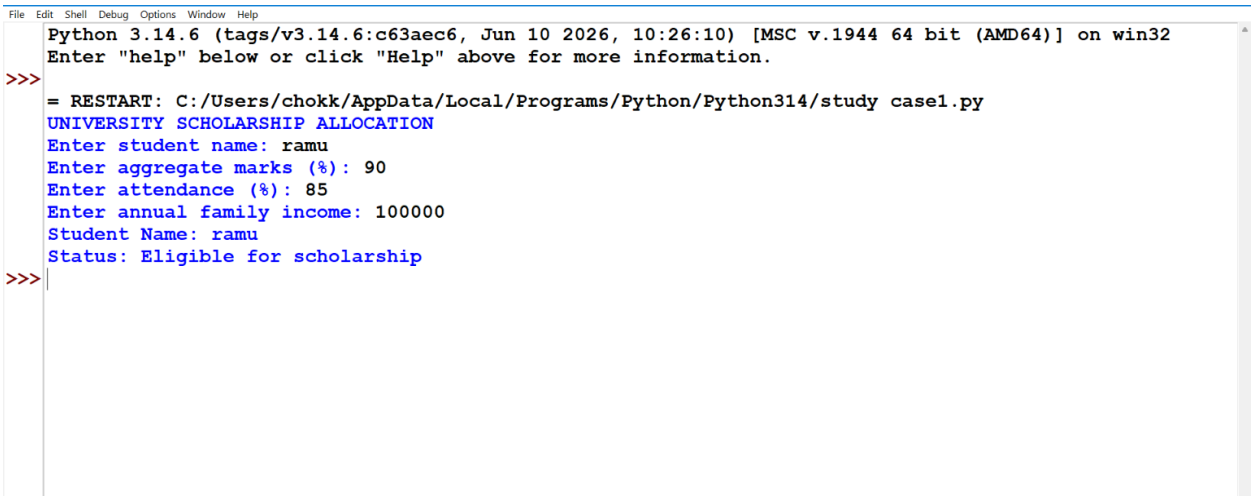
for s in eligible:

  print(s["name"], "Aggregate:", s["aggregate"],

    "Attendance:", s["attendance"],

    "Income:", s["income"])

6.Output Screenshot



7. Conclusion

The program checks scholarship eligibility based on marks, attendance, and family income. It displays eligible students in ranked order and demonstrates Python lists, dictionaries, loops, conditional statements, and sorting.

Program 2: Progressive Electricity Billing

1. Objectives

- To calculate electricity charges using progressive unit slabs.

- To include fixed charges, surcharge, and rebate.

- To alert the user when the bill increases significantly.

2. Algorithm

1. Start.

2. Enter units consumed and previous bill.

3. Calculate energy charges using the applicable unit slabs.

4. Add a fixed charge of ₹50.

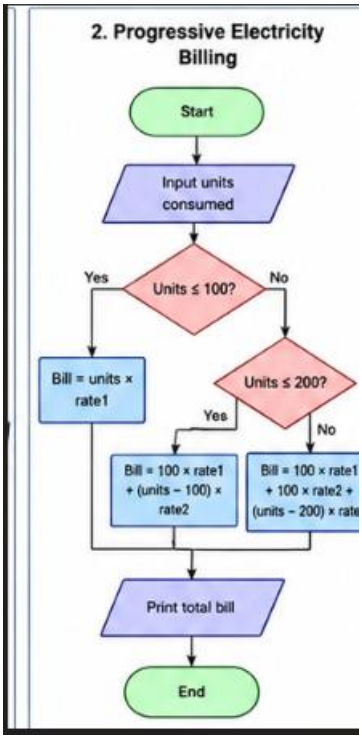
5. Add a 5% surcharge and apply a ₹30 rebate if energy charges are below ₹200.

6. Compare with the previous bill.

7. Display the total bill and any alert.

8. Stop.

3.flowchart



4. Pseudocode

START

INPUT units, previous_bill

IF units <= 100

  energy = units * 1.50

ELSE IF units <= 200

  energy = 150 + (units - 100) * 2.50

ELSE

  energy = 150 + 250 + (units - 200) * 4

END IF

surcharge = energy * 0.05

rebate = 30 IF energy < 200 ELSE 0

bill = energy + surcharge + 50 - rebate

DISPLAY bill

IF previous_bill > 0

  CALCULATE percentage increase

  IF increase > 30

    DISPLAY alert

  END IF

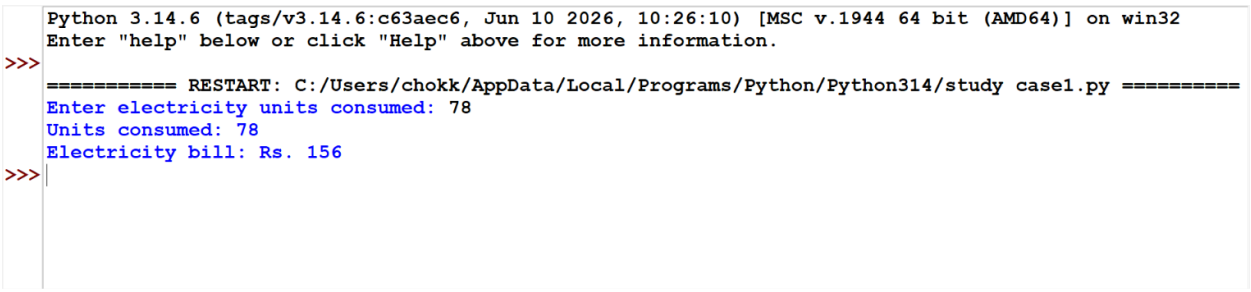
END IF

STOP

5. Python Code

units = int(input("Enter units consumed: "))previous = float(input("Previous bill amount (0 if none): "))if units <= 100: energy = units * 1.50elif units <= 200: energy = 150 + (units - 100) * 2.50else: energy = 150 + 250 + (units - 200) * 4surcharge = energy * 0.05rebate = 30 if energy < 200 else 0bill = energy + surcharge + 50 - rebateprint(f"Energy charge: Rs.{energy:.2f}")print(f"Surcharge: Rs.{surcharge:.2f}")print(f"Fixed charge: Rs.50.00")print(f"Rebate: Rs.{rebate:.2f}")print(f"Total bill: Rs.{bill:.2f}")if previous > 0: increase = (bill - previous) / previous * 100 print(f"Bill change: {increase:.1f}%") if increase > 30: print("Alert: Bill increased by more than 30%.")

6. Output Screen6shot



Rebate: Rs.0.00

Total bill: Rs.365.00

Bill change: 105.6%

Alert: Bill increased by more than 30%.

.

7. Conclusion

The program calculates an electricity bill using progressive unit rates and additional charges. It also compares the current bill with the previous bill and displays an alert for a large increase.

Program 3: Hospital Billing and Patient Classification

1. Objectives

- To calculate a patient's hospital bill.

- To include room charges, consultation fees, and test charges.

- To classify the bill amount.

2. Algorithm

1. Start.

2. Enter patient details, room type, number of days, consultation fee, and test charges.

3. Select the daily room rate.

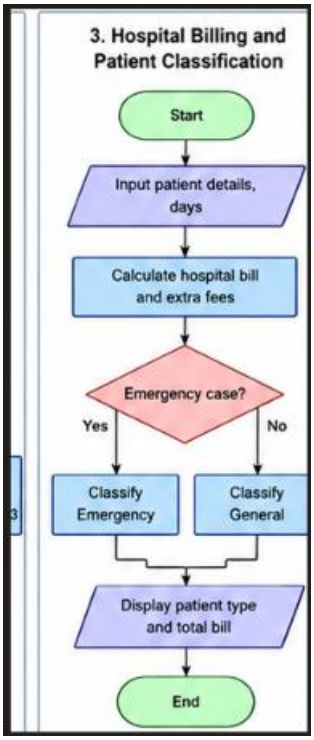
4. Calculate room charges and subtotal.

5. Apply a 10% discount if the subtotal is at least ₹10,000.

6. Classify the final bill.

7. Display the bill and stop.

3,flowchart



4. Pseudocode

START

INPUT name, room_type, days, consultation, tests

IF room_type = "private"

```
   rate = 2500
ELSE
   rate = 1000
END IF

room_charge = rate * days
subtotal = room_charge + consultation + tests

IF subtotal >= 10000
   discount = subtotal * 0.10
ELSE
   discount = 0
END IF

total = subtotal - discount

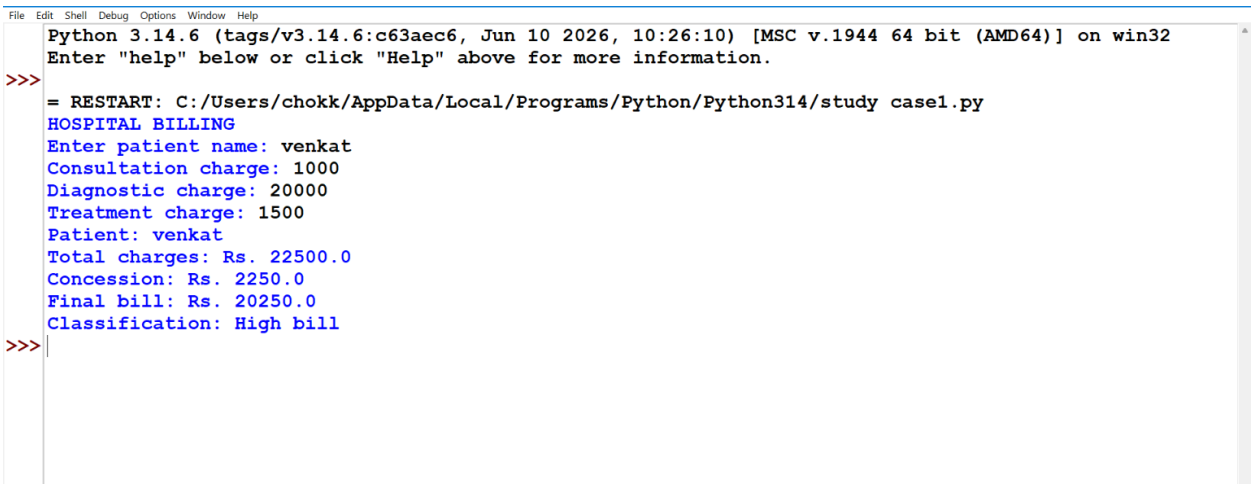
IF total < 5000
   category = "Basic"
ELSE IF total < 15000
   category = "Standard"
ELSE
   category = "High"
END IF

DISPLAY total, category
STOP
```

5. Python Code

name = input("Enter patient name: ")room = input("Room type (general/private): ").lower()days = int(input("Number of days: "))consultation = float(input("Consultation fee: Rs."))tests = float(input("Test charges: Rs."))if room == "private":   rate = 2500else: rate = 1000room_charge = rate * dayssubtotal = room_charge + consultation + testsdiscount = subtotal * 0.10 if subtotal >= 10000 else 0total = subtotal - discountif total < 5000:   category = "Basic"elif total < 15000:   category = "Standard"else: category = "High"print("\n--- HOSPITAL BILL ---")print("Patient:", name)print(f"Room charges: Rs.{room_charge:.2f}")print(f"Consultation: Rs.{consultation:.2f}")

6. Output Screenshot



7. Conclusion

The program calculates hospital charges and classifies patients' bills based on the final amount. It demonstrates arithmetic operations, conditions, and formatted output.

Program 4: Employee Payroll and Deduction Analysis

1. Objectives

- To calculate gross salary and deductions.

- To determine the employee's net salary.

- To display a basic payslip.

2. Algorithm

1. Start.

2. Enter employee name and basic salary.

3. Calculate HRA as 20% and DA as 10% of basic salary.

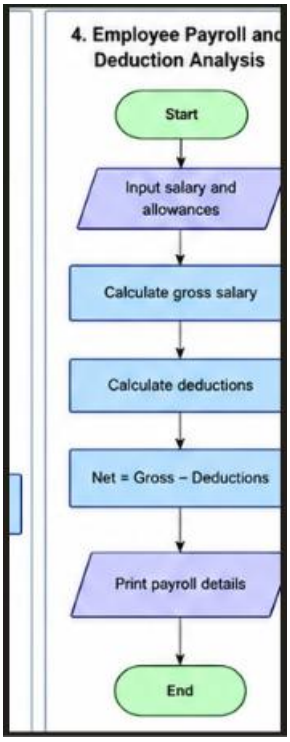
4. Calculate gross salary.

5. Calculate PF as 12% of basic salary and tax as 5% of gross salary.

6. Subtract deductions from gross salary.

7. Display the payslip and stop.

3. Flowchart



4. Pseudocode

START

INPUT name, basic_salary

hra = basic_salary * 0.20

da = basic_salary * 0.10

gross = basic_salary + hra + da

pf = basic_salary * 0.12

tax = gross * 0.05

deductions = pf + tax

net = gross - deductions

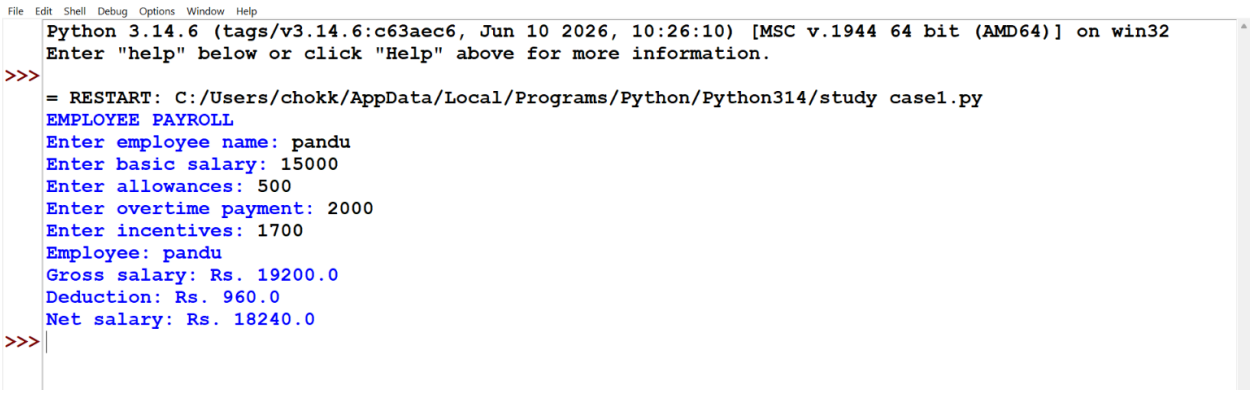
DISPLAY gross, deductions, net

STOP

5. Python Code

name = input("Enter employee name: ")basic = float(input("Enter basic monthly salary: Rs."))hra = basic * 0.20da = basic * 0.10gross = basic + hra + dapf = basic * 0.12tax = gross * 0.05deductions = pf + taxnet = gross - deductionsprint("\n--- EMPLOYEE PAYSLIP ---")print("Employee:", name)print(f"Basic salary: Rs.{basic:.2f}")print(f"HRA: Rs.{hra:.2f}")print(f"DA: Rs.{da:.2f}")print(f"Gross salary: Rs.{gross:.2f}")print(f"PF deduction: Rs.{pf:.2f}")print(f"Tax deduction: Rs.{tax:.2f}")print(f"Total deductions: Rs.{deductions:.2f}")print(f"Net salary: Rs.{net:.2f}")

6. Output Screenshot



7. Conclusion

The program calculates an employee's gross salary, deductions, and net salary. It demonstrates variables, arithmetic operators, and formatted output.

Program 5: Multi-Plan Telecommunications Cost Optimization

1. Objectives

- To calculate the cost of different mobile plans.

- To compare plans according to call minutes and data usage.

- To identify the cheapest plan.

2. Algorithm

1. Start.

2. Enter monthly call minutes and data usage.

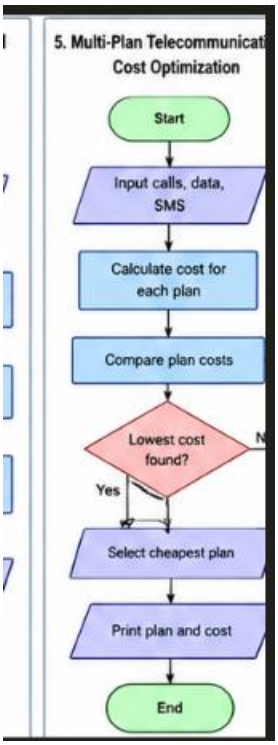
3. Calculate the cost of each plan, including excess-use charges.

4. Display the cost of every plan.

5. Identify the plan with the lowest cost.

6. Stop.

3. Flowchart



4. Pseudocode

START

INPUT minutes, data_gb

FOR each plan

  extra_minutes = MAX(0, minutes - included_minutes)

  extra_data = MAX(0, data_gb - included_data)

  cost = monthly_fee

    + extra_minutes * minute_rate

    + extra_data * data_rate

END FOR

DISPLAY all plan costs

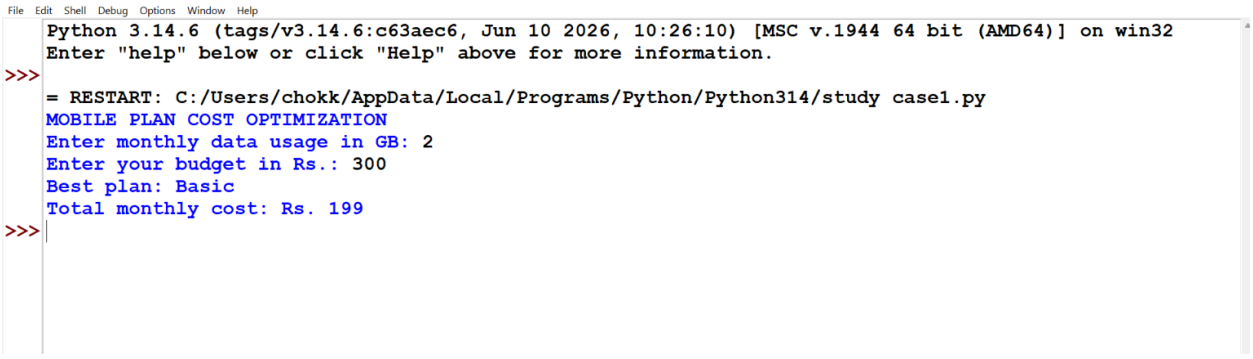
DISPLAY plan with minimum cost

STOP

5. Python Code

minutes = int(input("Enter monthly call minutes: "))data = float(input("Enter monthly data usage in GB: "))plans = { "Plan A": [199, 200, 2, 1.00, 50], "Plan B": [299, 500, 5, 0.50, 30], "Plan C": [399, 1000, 10, 0.25, 20]}costs = {}for name, p in plans.items(): fee, included_min, included_data, min_rate, data_rate = p extra_min = max(0, minutes - included_min) extra_data = max(0, data - included_data) cost = fee + extra_min * min_rate + extra_data * data_rate costs[name] = cost print(f"{name}: Rs.{cost:.2f}")best = min(costs, key=costs.get)print("Cheapest plan:", best, f"(Rs.{costs[best]:.2f})")

6. Output Screenshot



7. Conclusion

The program compares mobile plans based on usage and identifies the cheapest option. It demonstrates dictionaries, loops, arithmetic calculations, and finding minimum values.

Program 6: Railway Fare and Reservation Calculator

1. Objectives

- To calculate railway fare according to journey distance.

- To apply sample passenger-category discounts.

- To include reservation charges.

2. Algorithm

1. Start.

2. Enter passenger name, distance, category, and number of passengers.

3. Calculate base fare at ₹1.50 per kilometre per passenger.

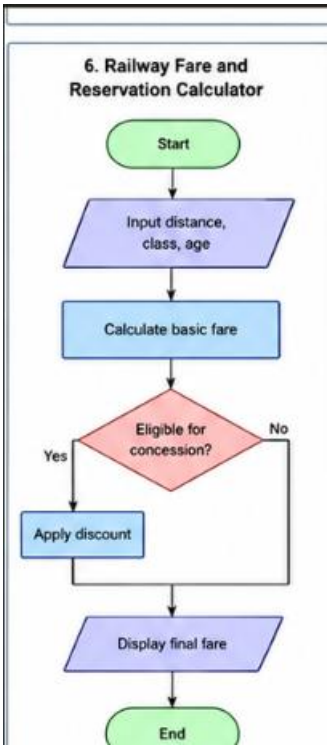
4. Apply the relevant sample discount.

5. Add ₹30 reservation fee per passenger.

6. Display the total fare.

7. Stop.

3. Flowchart



4. Pseudocode

START

INPUT name, distance, category, passengers

base = distance * 1.50 * passengers

IF category = "child"

  rate = 0.50

ELSE IF category = "senior"

  rate = 0.40

ELSE

  rate = 0

END IF

discount = base * rate

reservation = 30 * passengers

total = base - discount + reservation

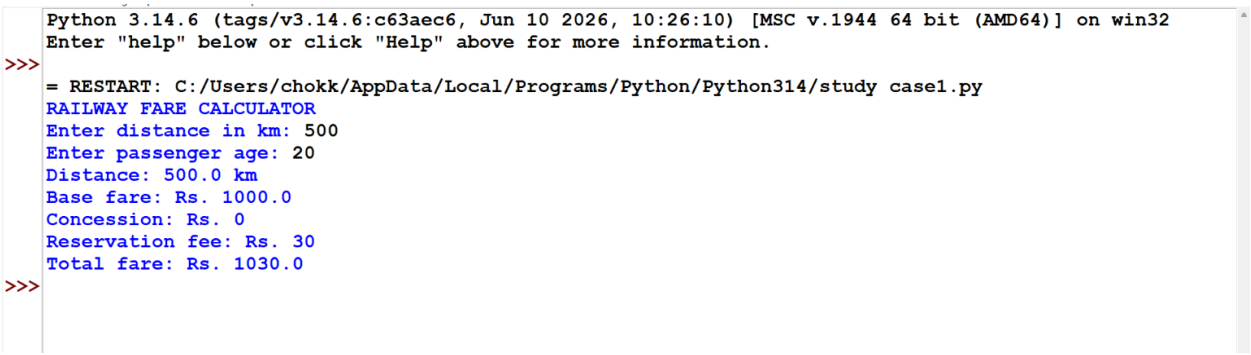
DISPLAY base, discount, reservation, total

STOP

5. Python Code

name = input("Enter passenger/group name: ")distance = float(input("Enter distance in km: "))category = input("Category (adult/child/senior): ").lower()passengers = int(input("Number of passengers: "))base = distance * 1.50 * passengersif category == "child": rate = 0.50elif category == "senior": rate = 0.40else: rate = 0discount = base * ratereservation = 30 * passengerstotal = base - discount + reservationprint("\n--- RAILWAY FARE --- ")print("Passenger/group:", name)print(f"Base fare: Rs.{base:.2f}")print(f"Discount: Rs.{discount:.2f}")print(f"Reservation fee: Rs.{reservation:.2f}")print(f"Total fare: Rs.{total:.2f}")

6. Output Screenshot



7. Conclusion

The program calculates a sample railway fare using distance, passenger category, and reservation fees. It demonstrates conditional statements and arithmetic operations.

Program 7: Retail Billing with Multiple Discount Rules

1. Objectives

- To calculate the total price of purchased items.

- To apply discounts based on the bill amount.

- To apply an additional membership discount.

2. Algorithm

1. Start.

2. Enter the number of item types.

3. Input each item's name, price, and quantity.

4. Calculate the subtotal.

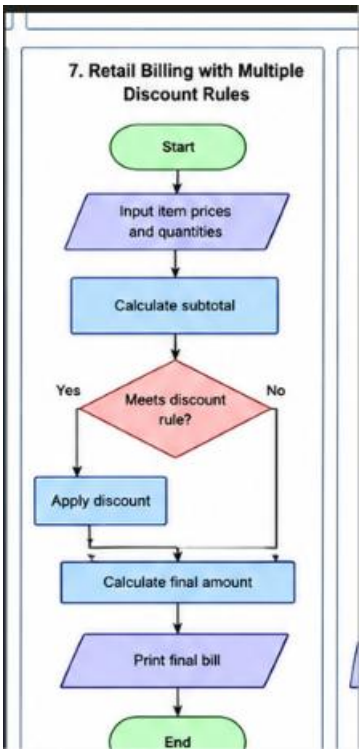
5. Apply a 10% discount if the subtotal is at least ₹5,000, or 5% if it is at least ₹2,000.

6. Apply an additional 5% membership discount if applicable.

7. Display the final bill.

8. Stop.

3. Flowchart



4. Pseudocode

```
START
INPUT number_of_items
subtotal = 0

FOR each item
   INPUT name, price, quantity
   subtotal = subtotal + price * quantity
END FOR

IF subtotal >= 5000
   discount_rate = 0.10
ELSE IF subtotal >= 2000
   discount_rate = 0.05
```

ELSE

discount_rate = 0

END IF

amount = subtotal - subtotal * discount_rate

INPUT membership_status

IF membership_status = "yes"

amount = amount - amount * 0.05

END IF

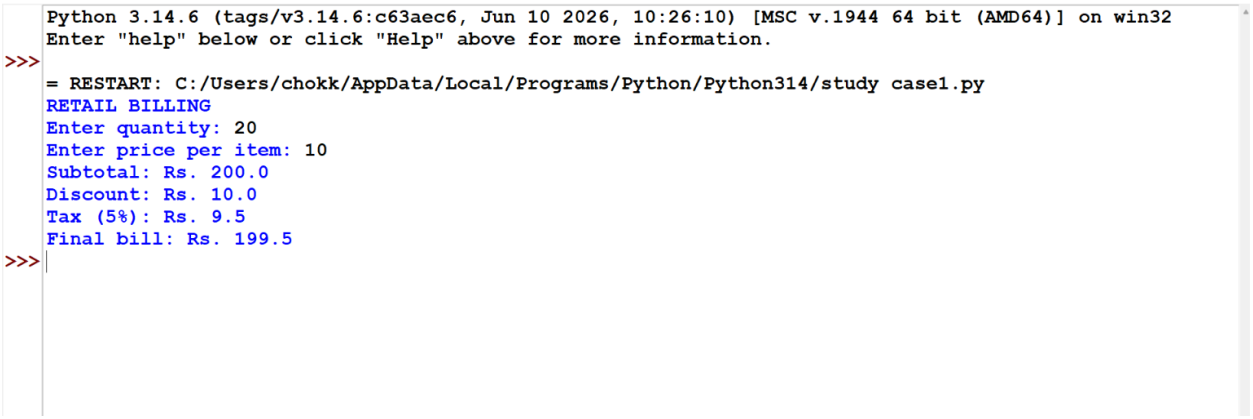
DISPLAY subtotal, discounts, amount

STOP

5. Python Code

items = []count = int(input("Enter number of item types: "))for i in range(count): name = input(f"Item {i + 1} name: ") price = float(input("Price per item: Rs.")) quantity = int(input("Quantity: ")) items.append((name, price, quantity))subtotal = sum(price * quantity for name, price, quantity in items)if subtotal >= 5000: rate = 0.10elif subtotal >= 2000: rate = 0.05else: rate = 0discount = subtotal * rateamount = subtotal - discountmember = input("Membership customer? (yes/no): ").lower()member_discount = amount * 0.05 if member == "yes" else 0total = amount - member_discountprint("\n--- RETAIL BILL ---")for name, price, quantity in items:

6. Output Screenshot



7. Conclusion

The program generates a retail bill and applies multiple discounts in sequence. It demonstrates lists, loops, conditions, and arithmetic calculations.

Program 8: Manufacturing Cost and Profitability Analysis

1. Objectives

- To calculate manufacturing cost per unit.

- To determine total production cost and revenue.

- To calculate profit or loss and profit margin.

2. Algorithm

1. Start.

2. Enter units produced, material cost, labour cost, overhead cost, and selling price.

3. Calculate unit cost.

4. Calculate total production cost and revenue.

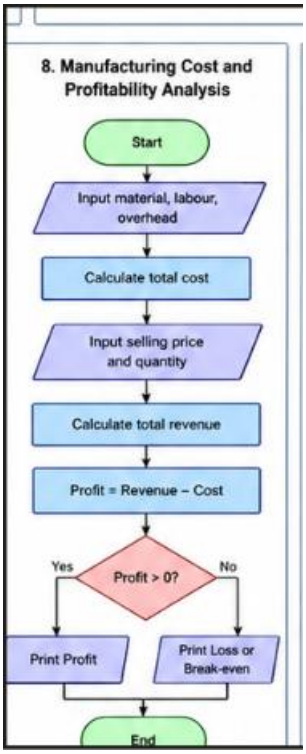
5. Calculate profit or loss.

6. Calculate profit margin.

7. Display the results.

8. Stop.

3. Flowchart



4. Pseudocode

START

INPUT units, material, labour, overhead, selling_price

unit_cost = material + labour + overhead

total_cost = unit_cost * units

revenue = selling_price * units

profit = revenue - total_cost

IF revenue > 0

  margin = profit / revenue * 100

ELSE

  margin = 0

END IF

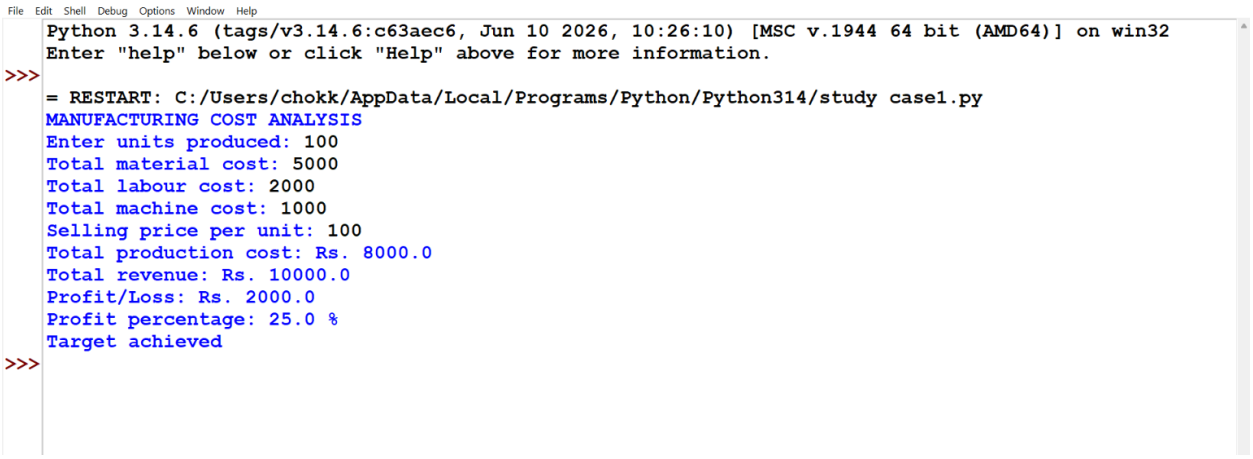
DISPLAY unit_cost, total_cost, revenue, profit, margin

STOP

5. Python Code

units = int(input("Units produced: "))material = float(input("Raw material cost per unit: Rs."))labour = float(input("Labour cost per unit: Rs."))overhead = float(input("Overhead cost per unit: Rs."))selling_price = float(input("Selling price per unit: Rs."))unit_cost = material + labour + overheadtotal_cost = unit_cost * unitsrevenue = selling_price * unitsprofit = revenue - total_costif revenue > 0:   margin = profit / revenue * 100else: margin = 0print("\n--- PROFITABILITY ANALYSIS ---")print(f"Cost per unit: Rs.{unit_cost:.2f}")print(f"Total production cost: Rs.{total_cost:.2f}")print(f"Total revenue: Rs.{revenue:.2f}")print(f"Profit/Loss: Rs.{profit:.2f}")print(f"Profit margin: {margin:.2f}%")

6. Output Screenshot



7. Conclusion

The program calculates manufacturing costs, revenue, profit, and profit margin. It helps understand how production expenses affect profitability.

Program 9: Multi-Currency Travel Budget Analysis

1. Objectives

- To convert foreign-currency expenses into Indian rupees.

- To compare travel expenses with the available budget.

- To identify remaining funds or overspending.

2. Algorithm

1. Start.

2. Enter the travel budget in INR.

3. Enter the expense amount in foreign currency.

4. Enter the exchange rate in INR per unit of foreign currency.

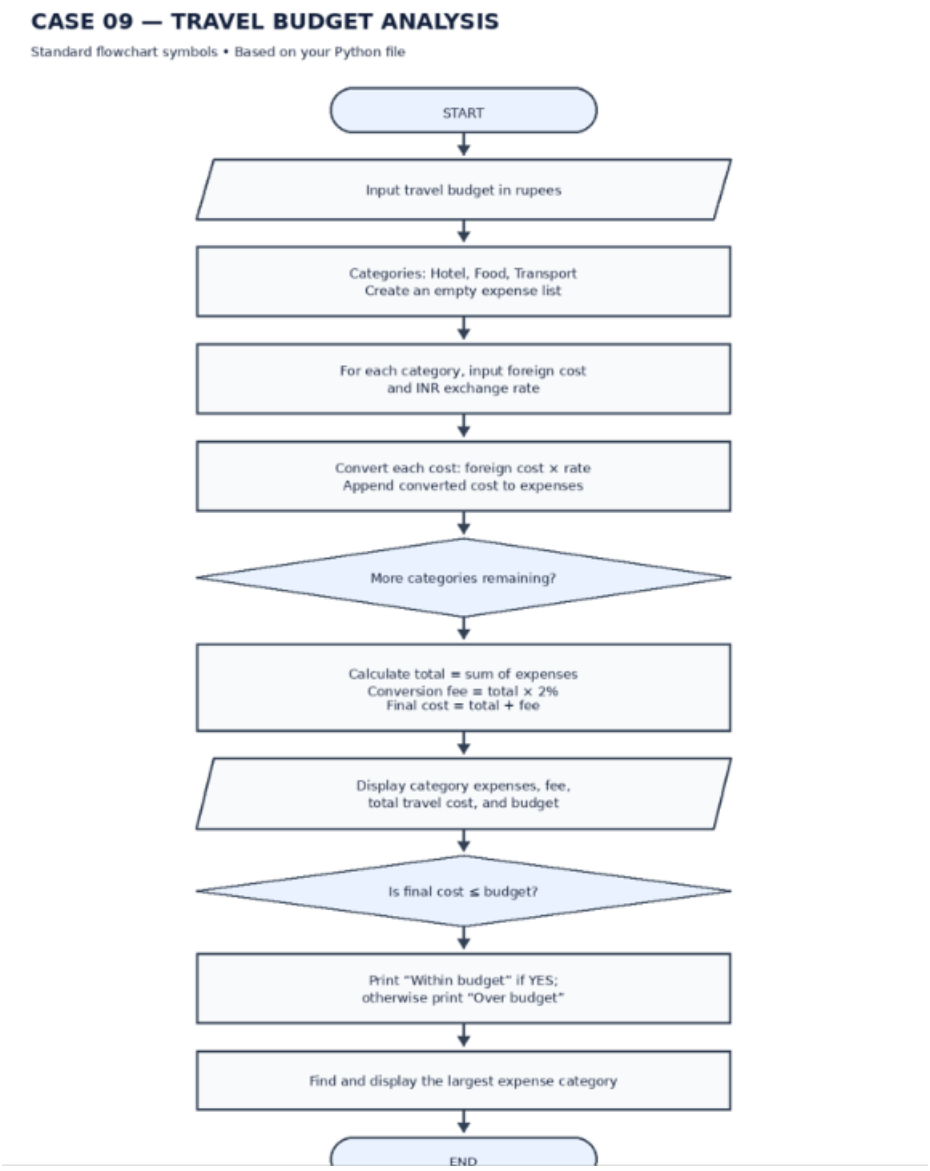
5. Convert the expense into INR.

6. Compare the converted expense with the budget.

7. Display the remaining budget or overspending.

8. Stop.

3. Flowchart



4. Pseudocode

START

INPUT budget, foreign_amount, exchange_rate

expense_inr = foreign_amount * exchange_rate

balance = budget - expense_inr

DISPLAY expense_inr

IF balance >= 0

  DISPLAY remaining budget = balance

ELSE

  DISPLAY overspending = ABS(balance)

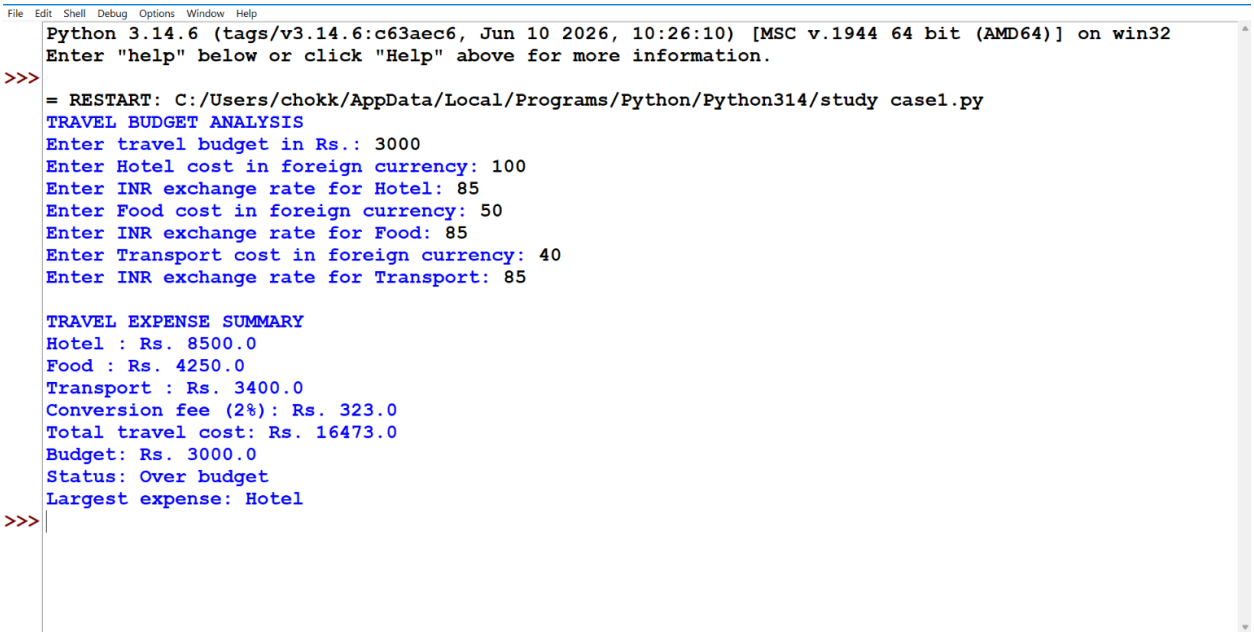
END IF

STOP

5. Python Code

budget = float(input("Travel budget in INR: Rs."))foreign_amount = float(input("Expense in foreign currency: "))currency = input("Currency name/code: ")rate = float(input(f"Exchange rate (INR per 1 {currency}): Rs."))expense_inr = foreign_amount * ratebalance = budget - expense_inrprint("\n--- TRAVEL BUDGET ---")print(f"Expense: {foreign_amount:.2f} {currency}")print(f"Converted expense: Rs.{expense_inr:.2f}")if balance >= 0: print(f"Budget remaining: Rs.{balance:.2f}")else: print(f"Budget exceeded by: Rs.{-balance:.2f}")

6. Output Screenshot



7. Conclusion

The program converts foreign-currency expenses into INR and checks whether the travel budget is sufficient. It demonstrates input, arithmetic operations, and conditional statements.

Program 10: Campus Event Financial Planning

1. Objectives

- To calculate the cost of organising a campus event.

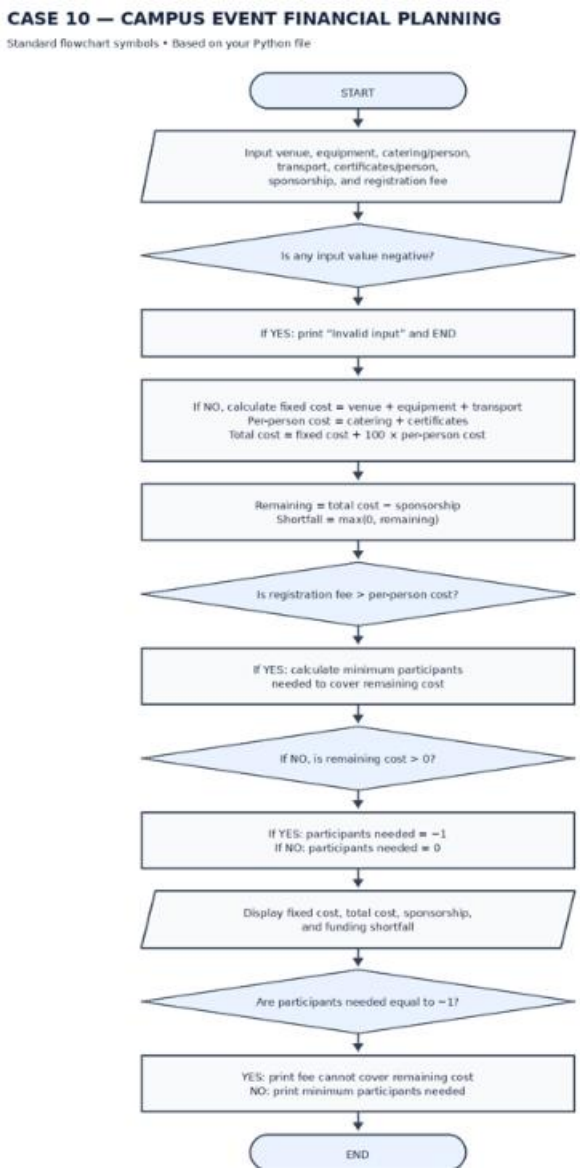
- To estimate ticket revenue and profit or loss.

- To calculate the number of tickets required to break even.

2. Algorithm

1. Start.
2. Enter venue, decoration, and other fixed costs.
3. Enter the variable cost per attendee, ticket price, and expected attendees.
4. Calculate fixed costs and total event costs.
5. Calculate expected ticket revenue.
6. Calculate profit or loss.
7. Calculate break-even tickets if the ticket price exceeds the variable cost per attendee.
8. Display the results and stop.

3. Flowchart



4. Pseudocode

START

INPUT venue, decoration, other_fixed

INPUT cost_per_person, ticket_price, attendees

fixed_cost = venue + decoration + other_fixed

total_cost = fixed_cost + cost_per_person * attendees

revenue = ticket_price * attendees

profit = revenue - total_cost

contribution = ticket_price - cost_per_person

IF contribution > 0

  break_even = CEILING(fixed_cost / contribution)

ELSE

  break_even = "Not possible"

END IF

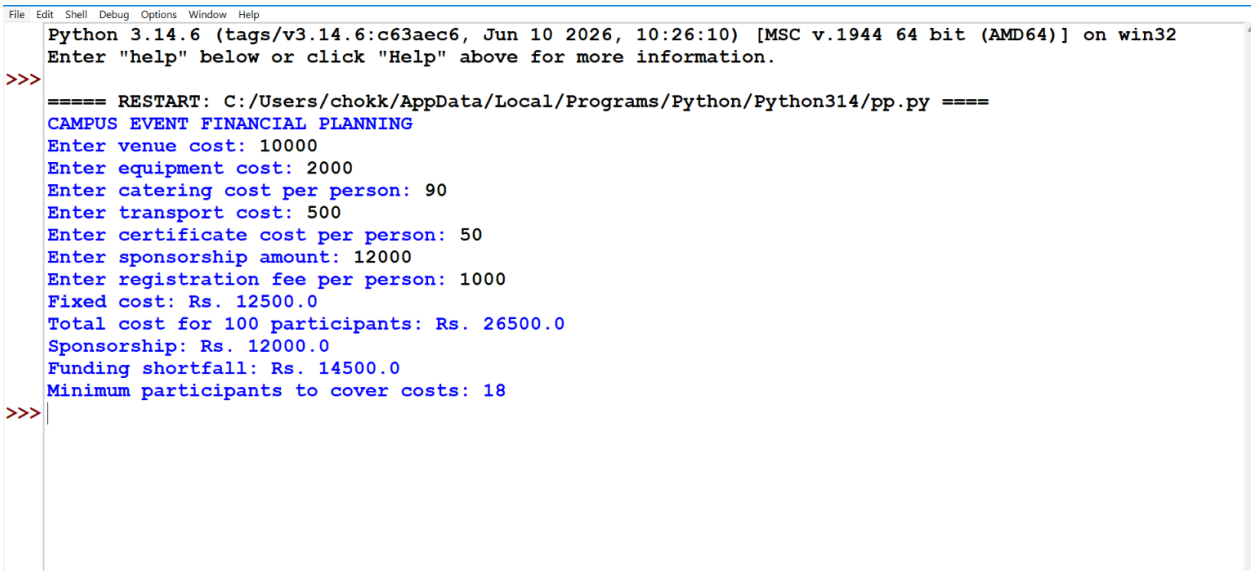
DISPLAY total_cost, revenue, profit, break_even

STOP

5. Python Code

venue = float(input("Venue cost: Rs."))decoration = float(input("Decoration cost: Rs."))other_fixed = float(input("Other fixed costs: Rs."))cost_person = float(input("Variable cost per attendee: Rs."))ticket_price = float(input("Ticket price per attendee: Rs."))attendees = int(input("Expected attendees: "))fixed_cost = venue + decoration + other_fixedtotal_cost = fixed_cost + cost_person * attendeesrevenue = ticket_price * attendeesprofit = revenue - total_costcontribution = ticket_price - cost_personprint("\n--- CAMPUS EVENT BUDGET ---")print(f"Fixed costs: Rs.{fixed_cost:.2f}")print(f"Total event cost: Rs.{total_cost:.2f}")print(f"Expected ticket revenue: Rs.{revenue:.2f}")print(f"Profit/Loss: Rs.{profit:.2f}")if contribution > 0:   break_even = int((fixed_cost + contribution - 1) // contribution)

6. Output Screenshot



7. Conclusion

The program calculates the costs, revenue, and profit or loss of a campus event. It also estimates the number of tickets needed to cover costs, helping organisers plan their budgets.